

شرح كامل لمشروع P2P Chat (المجموعة الرديفة)

مادة: برمجة التطبيقات الشبكية — المسار: Peer-to-Peer

هذا الملف يشرح مشروع المجموعة الرديفة كما استلمناه، بالإضافة إلى كل التعديلات والتحسينات التي أضفناها لمجموعتنا.

نظرة عامة على مشروع P2P

المشروع هو تطبيق دردشة جماعية وخاصة، لكن بنموذج Peer-to-Peer خالص — أي لا يوجد خادم مركزي. كل جهاز على الشبكة يلعب دورين بنفس الوقت:

- Server محلي: يستمع لاتصالات الأجهزة الأخرى الواردة إليه
 - Client: يتصل مباشرة بأي جهاز آخر يريد إرسال رسالة له
- الأجهزة تتعرف على بعضها من خلال جدول ثابت PEERS يربط كل اسم مستخدم بعنوان IP ومنفذ.

الفرق الجوهرى عن مشروعنا (Client-Server)

Client-Server (مشروعنا)	P2P (المجموعة الرديفة)
خادم مركزي واحد يدير كل شيء	لا يوجد — كل جهاز مستقل
كل عميل يتصل بالخادم فقط	كل جهاز يتصل مباشرة بكل جهاز آخر
فصل واضح بين دور الخادم ودور العميل	كل جهاز Server و Client بنفس الوقت

الملفات الأصلية لمشروعهم

- ملف مشروع_برمجة_تطبيقات.py — الكود الكامل (واجهة PyQt5 + منطق الشبكة)
- ملف PDF يشرح كل دالة بالتفصيل

جدول الأقران (PEERS) — قلب النظام

```
PEERS = {
    "Hamzah": ("192.168.30.130", 5000),
    "Ali": ("192.168.30.150", 5000),
    "Mohamed": ("192.168.30.140", 5000),
    "Maryam": ("192.168.30.141", 5000)
}
```

هذا القاموس هو قاعدة بيانات صغيرة: لكل اسم مستخدم، عنوان IP حقيقي للجهاز الخاص به. عندما يريد جهاز A إرسال رسالة للجهاز B، يبحث باسمه بهذا القاموس ليعرف IP و Port الذي يتصل به.

شرح الدوال الأساسية بأكودهم الأصلي

get_name_from_ip(ip)

عندما يستقبل الجهاز رسالة، لا يعرف فوراً اسم المُرسِل — يعرف فقط عنوان IP الخاص به. هذه الدالة تبحث بقاموس PEERS عن الاسم المرتبط بهذا IP، حتى تُعرَض الرسائل باسم صاحبها وليس بعنوانه.

receive_from_peer(name, sock, gui)

تعمل داخل Thread مستقل، وتستقبل الرسائل من جهاز واحد فقط بشكل مستمر. تُمَيِّز بين الرسائل الجماعية (تبدأ بـ [GROUP]) والرسائل الخاصة، وتعرض كلاً منها بالتبويب المناسب بالواجهة.

connect_to_peer(name, gui)

تُنشئ اتصال TCP مع جهاز معين، وتُعيد استخدام الاتصال إذا كان موجوداً مسبقاً (بدل فتح اتصال جديد بكل مرة). هذه الدالة هي الأساس الذي تعتمد عليه كل عمليات الإرسال.

listen_thread(gui, my_port)

تُحوّل الجهاز إلى خادم صغير يستمع للاتصالات الواردة من بقية الأجهزة، وتفتح Thread جديداً لكل اتصال وارد. بدون هذه الدالة، الجهاز لن يستطيع استقبال أي رسالة من الآخرين.

الواجهة (ChatGUI) — مبنية بـ PyQt5

- تبويب Group Chat: لعرض وإرسال الرسائل الجماعية
- تبويب Private Chat: لاختيار مستخدم معين وإرسال رسالة خاصة له فقط
- دوال log_group / log_private: تعرض الرسائل بألوان مختلفة حسب المُرسِل

لماذا Threading هنا بالتحديد؟ (3 أنواع Threads)

1. Thread الاستماع (Server Thread): ينتظر الاتصالات الواردة بشكل دائم
2. Thread استقبال لكل قرين (Peer): واحد مستقل لكل اتصال نشط، يستقبل رسائله بدون التأثير على الباقين
3. Thread الواجهة الرسومية (PyQt5 Main Loop): يعمل بشكل مستقل لإدارة النوافذ والأزرار

المشاكل التي وجدناها بالكود الأصلي

المشكلة	الوصف
تعارض بيانات (Race Condition)	القاموس connections يُعَدّل من عدة Threads بدون أي قفل (Lock)، قد يتسبب بأخطاء أو تلف بيانات
عنونة ثابتة (Hardcoded)	جدول PEERS مكتوب مباشرة بالكود، أي تغيير IP يتطلب تعديل الكود وإعادة التشغيل
معالجة غير واضحة للمجهول	لو اتصل أكثر من جهاز غير مسجّل، كلاهما يحملان نفس الاسم بالقاموس، فيحدث تضارب
غياب السجل (History)	لا يوجد أي ملف لحفظ الرسائل؛ تُفقد بالكامل عند إغلاق البرنامج
إرسال جماعي غير متوازي	إرسال الرسالة الجماعية يتم بشكل متتابع (Sequential)، فعمل بطيء يؤخّر الجميع
غياب Timeout	لا يوجد أي مهلة محددة عند الاتصال، فجهاز غير متاح قد يُجمّد الواجهة

التعديلات التي طبّقناها (p2p_chat_improved.py)

إضافة Lock لحماية connections

```
connections_lock = threading.Lock()
```

```
with connections_lock:  
    connections[name] = sock
```

كل قراءة أو كتابة على connections أصبحت محاطة بهذا القفل، فلا يدخل أكثر من Thread واحد لتعديله بنفس اللحظة.

جدول أقران ديناميكي (peers.json)

بدل القاموس الثابت بالكود، أصبح PEERS يُقرأ من ملف peers.json خارجي، يُنشأ تلقائياً إذا لم يكن موجوداً. أي تغيير IP أصبح بتعديل الملف فقط، بدون لمس الكود.

معالجة أوضح للأجهزة غير المعروفة

بدل إرجاع نص ثابت "مستخدم غير معروف"، أصبحت الدالة تُرجع {ip: Unknown} — كل جهاز مجهول يحصل على مفتاح مميز خاص به، فلا يحدث تضارب.

سجل دائم للرسائل + تبويب History

أضيفت دالة log_message() تسجل كل رسالة (جماعية أو خاصة) بملف p2p_chat_history.log مع توقيت دقيق، وأضيف تبويب جديد بالواجهة لعرض آخر 100 رسالة من هذا الملف.

إرسال جماعي متوازي (Threads مستقلة)

```
for name in PEERS:
    if name == MY_NAME:
        continue
    threading.Thread(
        target=self._send_group_to_one,
        args=(name, msg),
        daemon=True
    ).start()
```

بدل إرسال الرسائل بشكل متتابع، أصبح كل قرين يُرسل له برسالته عبر Thread مستقل، فلا يُؤخر جهاز بطيء بقية العملية.

Timeout عند الاتصال + رسائل خطأ واضحة

```
sock.settimeout(3)
sock.connect((ip, port))
```

أصبح النظام يميز بوضوح بين 3 حالات فشل: انتهاء المهلة (socket.timeout)، رفض الاتصال (ConnectionRefusedError)، أو خطأ آخر — برسائل مختلفة لكل حالة.

إغلاق نظيف لكل الاتصالات

أضيفت دالة closeEvent() بالواجهة، تُغلق كل الاتصالات المفتوحة بشكل صحيح عند إغلاق البرنامج، بدل تركها مفتوحة.

نتائج الاختبار التي أجريناها

- ✓ تحميل peers.json وإنشاؤه تلقائياً عند عدم وجوده
- ✓ get_name_from_ip يتعامل بشكل سليم مع عناوين غير معروفة
- ✓ تسجيل الرسائل بملف p2p_chat_history.log يعمل بدقة
- ✓ إرسال رسالة خاصة بين جهازين فعلياً (محاكاة loopback) ووصلت بنجاح
- ✓ الواجهة الرسومية تُبنى بدون أخطاء وتعرض الرسائل بالتبويبات الصحيحة

أسئلة وأجوبة متوقعة عن مشروع P2P

كيف يعرف الجهاز اسم من يرسل له الرسالة بدون خادم مركزي؟	عبر جدول PEERS الذي يربط كل اسم بعنوان IP و Port. عند استقبال رسالة، تُستخدم دالة <code>get_name_from_ip</code> لمعرفة اسم المرسل من عنوانه.
ليش كل جهاز فيه Server و Client بنفس الوقت؟	لأن هذا هو تعريف P2P بالضبط — لا يوجد دور ثابت لجهاز واحد، كل جهاز يحتاج أن يستمع (Server) ويرسل (Client) بنفس الوقت.
شو أهم مشكلة لقيتوها بكودهم؟	غياب Lock عند تعديل القاموس المشترك connections من عدة Threads بنفس الوقت — وهي مشكلة Race Condition كلاسيكية.
ليش حوّلتوا PEERS لملف JSON؟	لجعل تغيير عناوين الأجهزة لا يتطلب تعديل الكود المصدري وإعادة تشغيل التطبيق — فقط تعديل الملف الخارجي.
هل غيّرتوا فكرة مشروعهم؟	لا، حافظنا على نفس البنية P2P بالكامل، فقط صحّحنا الأخطاء وحسّنا الموثوقية وتجربة الاستخدام دون تغيير المفهوم الأساسي.
كيف اختبرتموا التعديلات؟	بمحاكاة اتصال حقيقي بين جهازين (نفس الجهاز، منفذين مختلفين عبر loopback)، وتأكدنا أن الرسائل الخاصة والجماعية تصل بنجاح مع القفل الجديد.